MODULE 21 · DOMAIN 8 - HCP TERRAFORM

HCP Terraform: Workflows and Remote Operations

Three ways to start a run, three places it can execute, and what you lose by choosing the wrong one.

DOMAIN 8 of 8 COURSE WEIGHT 12% MODULE 21 of 27

admin@sthithapragna.com

HCP Terraform is not just a place to keep state. It is a place where **runs execute**, and most of its features depend on that.

What HCP Terraform adds

- **Managed state**, versioned and encrypted, with locking, and no backend for you to operate.
- **Remote execution**, in a consistent environment rather than on whichever machine ran the command.
- **Secure variable storage**, including secrets, held by the service rather than in files or a pipeline.
- **Run history**: who ran what, when, with which plan, and who approved it.
- **Governance features** layered on the run: policy enforcement, notifications, run tasks. Module twenty-three.

> ON THE EXAM
>
> Every item after the first depends on the run happening remotely. State management is the only one that survives local execution.

The cloud block

- **A cloud block inside the terraform block**, naming an organisation and a workspace. Terraform 1.1 onwards.
- **It replaces the older remote backend**, and cannot coexist with a backend block. Module sixteen.
- **No interpolation**, exactly like a backend block. Partial configuration works the same way.
- Workspaces can be selected by **name**, or by **tags** so one configuration maps to several.
- **terraform init** connects the working directory to the service, as it would any backend.

> ON THE EXAM
>
> cloud and backend are mutually exclusive. With HCP Terraform you configure no backend at all, because the service manages state.

Authenticating

- **terraform login** opens a browser, you authorise, and a token is stored in your CLI configuration.
- The token lands in a credentials file under your home directory, not in the working directory.
- For automation, set the token in an **environment variable** named after the host instead.
- **The token is a credential.** It goes in a secret store, never in a repository.
- **terraform logout** removes it again.

> ON THE EXAM
>
> Interactive login for a person, an environment variable for a pipeline. The same split as every other credential in this course.

The three workflows

WORKFLOW	HOW A RUN STARTS
UI and VCS-driven	A push to the connected repository, or a button. HashiCorp calls this the primary mode
CLI-driven	terraform plan or apply from your terminal, executing remotely
API-driven	Your own tooling uploads a configuration and starts a run

ON THE EXAM

All three produce the same kind of run in the same environment. What differs is what triggers it.

VCS-driven or CLI-driven?

VCS-DRIVEN

The repository is the source of truth

- Runs trigger on push automatically
- Speculative plans on pull requests
- Nobody applies from a laptop
- Configuration comes from a branch

CLI-DRIVEN

The engineer starts the run

- Familiar terraform plan and apply
- Output streams to the terminal
- Fits an existing CI/CD pipeline
- Good first step when migrating

THE TIEBREAKER

The deciding question is what should trigger a run: a commit, or a person. Both execute in the same place.

The three execution modes

MODE	WHERE THE RUN HAPPENS
Remote	On HCP Terraform's infrastructure. The default for a new workspace
Local	On your workstation. The workspace stores state only
Agent	On a lightweight agent you run, inside private or on-premises infrastructure

ON THE EXAM

Workflow is how a run starts. Execution mode is where it runs. Two independent settings, frequently confused.

What Local execution mode costs you

- **The workspace acts only as a remote backend for state.** That is HashiCorp's own description.
- **Many features rely on remote execution and are not available**: policy enforcement, cost estimation, notifications.
- **Workspace variables and variable sets are not evaluated at all** in local execution mode.
- So values must come from your own environment, which is exactly what the service was meant to centralise.
- It remains legitimate for **debugging**, or where a run genuinely must happen on a particular machine.

> ON THE EXAM
>
> Choosing Local silently disables most of what HCP Terraform is for. Nothing errors; the features simply stop applying.

What a remote run actually does

- Terraform runs on HCP Terraform's infrastructure, and the **progress streams to your terminal**.
- HashiCorp's aim is an experience **equivalent to local operations**, including the confirmation prompt.
- **The service always plans first, then applies that plan's output.** The division is enforced, not optional.
- Variables and credentials come from the **workspace**, so your laptop needs no cloud credentials at all.
- **Saved plans work remotely** too, for workspaces without a linked repository, from CLI v1.6.

> ON THE EXAM
>
> No cloud credentials on the workstation is the quiet security benefit. The engineer authenticates to HCP Terraform, and nothing else.

Agents

- **A lightweight agent you run inside your own network**, which polls HCP Terraform for work.
- For infrastructure the service **cannot reach**: private networks, on-premises systems, isolated environments.
- **The connection is outbound**, so no inbound access to your network is required.
- You keep policy enforcement, run history and variable management, unlike local execution mode.
- It is a paid feature rather than part of the free offering. Module twenty-three covers editions.

> ON THE EXAM
>
> Agent mode is the answer for unreachable infrastructure. Local mode is not, because it gives up the governance features too.

Dynamic credentials

- **Short-lived credentials issued per run**, rather than long-lived keys stored as workspace variables.
- HCP Terraform authenticates to your cloud, which trusts it, and the credential expires with the run.
- **Nothing long-lived is stored**, so there is no key to leak, rotate or forget.
- This is module two's advice and module thirteen's ephemerality, provided as a managed feature.
- It requires remote execution, since the credential is issued to the run environment.

> ON THE EXAM
>
> Storing a long-lived access key as a workspace variable works and is the weaker option. Dynamic credentials remove the secret entirely.

A workspace created by init

- **Name a workspace that does not exist and init creates it**, telling you it has done so.
- **No variables are created by default**, unless your organisation has a global variable set.
- **HCP Terraform will use *.auto.tfvars files if present**, but workspace-specific values usually still need setting.
- **Execution mode defaults to Remote**, and the workspace is not connected to any repository.
- HashiCorp's warning: an automatically created workspace **might not be immediately ready to use**.

> ON THE EXAM
>
> A successful init and a working workspace are different things. The first apply often fails on missing variables.

Migrating to HCP Terraform

- **Replace the backend block with a cloud block**, then run terraform init.
- Terraform offers to **migrate existing state** into the workspace, as with any backend change.
- **Back up your state first.** Module sixteen's advice applies unchanged.
- Set workspace variables **before the first run**, since the configuration no longer reads your local environment.
- The **CLI-driven workflow is the gentlest first step**, because the commands you type do not change.

> ON THE EXAM
>
> The migration is a backend migration with extra setup. What changes afterwards is where credentials and variables live.

Reading the scenario

THE WORDING SAYS	IT IS POINTING AT
Policies stopped being enforced	Execution mode was set to Local
The repository should trigger runs	The VCS-driven workflow
Infrastructure we cannot reach from the internet	An agent, not local execution mode
Engineers should not hold cloud credentials	Remote execution, and dynamic credentials
The first apply failed on missing variables	An auto-created workspace has none
Custom tooling starts our deployments	The API-driven workflow

ON THE EXAM

Separate the two questions: what starts the run, and where it executes. Most confusion here is one being read as the other.

> An organisation runs Sentinel policies on every workspace to enforce tagging and instance size limits. One team found remote runs slow to debug and changed their workspace's execution mode to Local.
>
> Three months later an audit finds that team's workspace has deployed resources violating two policies. No policy failures were recorded, and no runs errored.

What happened?

A The policies were misconfigured and silently passed everything

B Local execution mode disables remote operations, and policy enforcement depends on them

C Sentinel policies apply only to VCS-driven workspaces

D The team used -auto-approve, which bypasses policy checks

◆ PAUSE AND DECIDE

What happened?

A Misconfigured policies

They worked everywhere else in the organisation. One workspace differing points at a workspace setting, not at the policies.

B Local disables remote operations

Correct. HashiCorp lists policy enforcement, cost estimation and notifications as features requiring remote execution.

C VCS-driven only

Policies apply across workflows. What matters is the execution mode, not what triggered the run.

D -auto-approve bypasses

It skips the interactive confirmation, not the policy stage. A policy failure would still stop the run.

THE SIGNAL

Nothing errored, which is the tell. The policies were not failing; they were never being evaluated.

> A team manages Kubernetes clusters that sit in a private data centre with no inbound internet access and no route from HCP Terraform's runners. They want policy enforcement, run history and centrally managed variables.
>
> One engineer suggests setting execution mode to Local so runs happen on a jump host inside the network.

What should be used instead, and why?

A Local execution mode is correct. It is the only way to reach private infrastructure

B An HCP Terraform agent, which runs inside the network and keeps the governance features

C Open inbound firewall rules so HCP Terraform's runners can reach the clusters

D The API-driven workflow, which does not require network access

◆ PAUSE AND DECIDE

What should be used instead, and why?

A Local mode

Would reach the clusters and would give up the three things they explicitly asked for. Scenario 1 is this decision playing out.

B An agent

Correct. The agent runs inside the network with an outbound connection, and runs remain HCP Terraform runs with policies, history and variables intact.

C Open inbound rules

Solves a networking constraint by removing it, on a private data centre, which is the opposite of what agents exist to avoid.

D API-driven

Confuses workflow with execution mode. An API-driven run still executes somewhere, and by default that is HCP Terraform's infrastructure.

THE SIGNAL

Local and Agent both move execution off HCP Terraform's runners. Only the agent keeps the run under HCP Terraform's control.

> An engineer adds a cloud block naming a workspace that does not exist yet and runs terraform init. Terraform reports that it created the workspace, and init succeeds.
>
> She immediately runs terraform apply. The run starts remotely and fails, reporting that a required variable has no value. The same configuration applies without issue on her machine.

What is the explanation?

A. The workspace is still being provisioned and needs a few minutes

B. A new workspace has no variables. Her local environment supplied them before, and remote runs do not see it

C. The cloud block does not support variables and she must switch to a backend block

D. terraform init must be run a second time to synchronise variables

◆ PAUSE AND DECIDE

What is the explanation?

A Still provisioning

HashiCorp does warn a new workspace 'might not be immediately ready', meaning settings need configuring - not that time alone fixes it.

B No variables, and no local environment

Correct. No variables are created by default, and a remote run executes on HCP Terraform's infrastructure with no sight of her shell.

C cloud does not support variables

Workspace variables are a core feature. The cloud block itself takes no interpolation, which is a different restriction.

D Run init twice

Invents a synchronisation step. Init connects the directory to the workspace; it never uploads environment variables.

THE SIGNAL

The planted detail is that it works locally. That is what proves the value was coming from her environment all along.

> A regulated team wants their Git repository to be the single source of truth. Every change must be reviewed as a pull request with a plan attached, and nobody should be able to apply from a workstation.
>
> They currently use the CLI-driven workflow with remote execution.

What should they change?

A. Nothing. Remote execution already prevents applying from a workstation

B. Move to the VCS-driven workflow, so runs trigger from the repository and pull requests get speculative plans

C. Switch execution mode to Agent, which enforces repository-based runs

D. Move to the API-driven workflow, which is the only one supporting approvals

◆ PAUSE AND DECIDE

What should they change?

A Nothing needed

Remote execution changes where a run happens, not who can start one. An engineer can still type terraform apply and trigger a real run.

B VCS-driven workflow

Correct. Runs trigger from the repository, pull requests receive speculative plans, and the branch becomes the source of truth.

C Agent mode

Execution mode again, answering a question about triggering. An agent changes where a run executes and nothing about what starts it.

D API-driven only

Invents an exclusivity. Approvals are available across workflows, and the API-driven route would mean building tooling for no reason.

THE SIGNAL

Option A is the sharp one. Remote execution moves the run off the laptop; it does not stop the laptop starting it.

Module 21 recap

- **Three workflows:** UI and VCS-driven, CLI-driven, API-driven. What starts a run.
- **Three execution modes:** Remote, Local, Agent. Where it runs.
- **Local mode makes the workspace a state store only**, disabling policies and variables.
- **Agent mode keeps the governance features** while running inside your network.
- **The cloud block replaces the backend block** and takes no interpolation.
- **A new workspace has no variables**, and remote runs cannot see your local environment.

> ON THE EXAM
>
> Remote execution means your workstation holds no cloud credentials. Dynamic credentials go further and store no key at all.